NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY, KHULNA

Final Project Report

AI Smart Campus System

Professional Academic Report

Course: CSE 4204 - Mobile Computing Lab

Week 12 Final Submission

Prepared by: [Team member names and student IDs to be completed]

Supervisor: [Supervisor name to be completed]

Submission date: 22 October 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Methodology
5. System Architecture
6. Database Design
7. Technologies Used
8. Features
9. AI Integration
10. Testing and Results
11. Limitations and Future Work
12. Conclusion and References

1. Introduction

AI Smart Campus System is a web-based academic management and student-success platform developed for Northern University of Business and Technology, Khulna (NUBTK). The reviewed repository combines a React single-page frontend, Laravel REST API, MySQL persistence, role-based access control, academic operations, notices, exports and server-side AI-assisted workflows.

2. Problem Statement

Academic administration often separates account approval, course records, attendance, assessment, notices, routines, service requests and early risk identification. This project addresses that fragmentation through a single authenticated workspace with student, faculty and administrator scopes. It keeps API credentials and authoritative academic context on the server.

3. Objectives

Objective	Evidence in implementation
Centralize academic workflows	Dashboards, academic management, notices, campus operations, exports and profiles.
Enforce role-aware access	Sanctum, approval middleware, policies and controller checks.
Support students	Tasks, resources, recommendations, monitoring and risk alerts.
Provide advisory AI	Gemini assistant/recommendations and OpenAI risk-analysis integration.

4. Methodology

The system uses a layered web approach. React components call Axios API services. Laravel validates input, authenticates tokens, applies approval and authorization checks, performs database operations and invokes external AI providers only from the server. Laravel migrations version the schema. PHPUnit, Vitest and Playwright test suites are present.

5. System Architecture

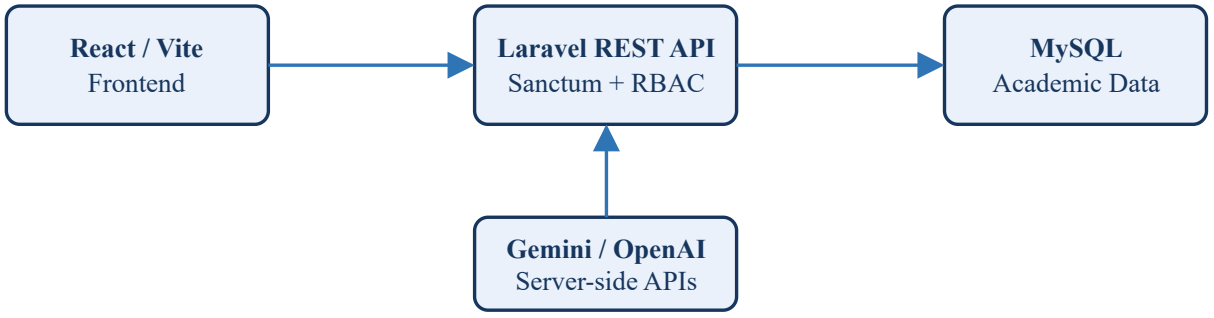


Figure 1. Layered architecture. API credentials remain in the backend environment, not the browser.

6. Database Design

The authoritative schema is the Laravel migration set. Core entities include users, students, faculty, courses, course enrollments, attendance records, academic records, performance metrics, notices and notice reads, tasks, learning resources, recommendations, risk alerts, AI conversations/messages, departments and campus-operation tables.

Domain	Representative tables
Identity and access	users, personal_access_tokens, students, faculty, departments
Academic	courses, course_enrollments, attendance_records, assessment_records, academic_records, performance_metrics, course_schedules
Communication	notices, notice_reads, campus_tasks, learning_resources, recommendations
AI and monitoring	risk_alerts, ai_conversations, ai_messages
Operations	exams, events, fees, tickets, leaves, reschedules, books and loans

7. Technologies Used

Layer	Technologies evidenced
Frontend	React 19, React Router, Vite, Axios, Bootstrap, Material UI, React Markdown
Backend	PHP 8.2+, Laravel 12, Laravel Sanctum
Database	MySQL in the default environment configuration
AI	Google Gemini Generate Content API; OpenAI Responses API
Testing	PHPUnit, Vitest, Testing Library, Playwright
Deployment	Composer, npm, queues/scheduler, HTTPS/CORS guidance

8. Features

Students can register, await approval, manage profile data and photos, view dashboards, export transcript and attendance information, use tasks/resources/notices/campus services, obtain recommendations and use the conversational assistant. Faculty have assigned-course workspaces, record entry, scoped monitoring, notices, services and assistant access. Administrators manage approval, departments, courses, notices, operations and institution-wide monitoring.

9. AI Integration

The Gemini assistant uses a server-side system prompt, recent history, authenticated context selection, timeouts and retry handling. Course recommendation determines candidates locally and can fall back to deterministic ranking. OpenAI risk analysis uses a strict JSON schema. In the current local implementation, configured-provider failure can produce an explicitly labelled baseline result derived from verified attendance, CGPA and missed-class indicators; missing configuration remains a clear error.

10. Testing and Results

The repository Final Testing Report dated 18 October 2026 records 93 automated tests passed and 0 failed: 10 frontend tests and 83 Laravel tests with 423 assertions. It states that AI calls were mocked, so this validates integration behavior and error paths rather than live provider availability. A focused StudentMonitoringTest run after the baseline fallback change passed 5 tests and 30 assertions.

11. Limitations and Future Work

Team names, student IDs and supervisor details were not available and are marked as placeholders. No current submission screenshots were present. Live AI depends on credentials, quota and network availability; mocked tests are not a live-provider demonstration. The reviewed code is a web application, not a native mobile package. Future work includes CI gates, live-provider monitoring, accessibility audit, sanitized responsive screenshots and usability evaluation.

12. Conclusion

The system demonstrates an integrated, role-aware academic management solution with migration-managed data, protected REST APIs and server-side AI integration. Its design emphasizes authorization, validation, provider isolation and graceful fallback.

References

[1] Project repository: README.md, backend/routes/api.php, migrations, controllers, services and tests, reviewed 22 October 2026. [2] Project documentation/API.md, ER_DIAGRAM.md, DEPLOYMENT.md and testing-evidence/Final_Testing_Report.md. [3] Laravel 12 and Sanctum

documentation. [4] React documentation. [5] Google Gemini API documentation. [6] OpenAI Responses API documentation.